Ho Chi Minh City University of Science

Faculty of Information Technology

Lab 01 Report

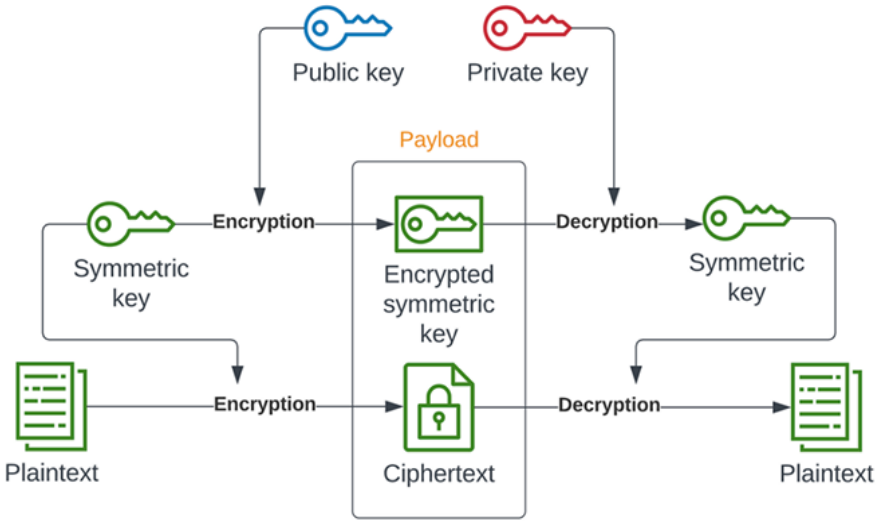
Hybrid Encryption

Student: 22127233 – Tran Hoang Linh

November 21, 2025

1 Introduction

In this lab, I implemented a hybrid encryption system to securely exchange files between a sender and a receiver. The system employs symmetric encryption (AES) to protect the file contents and asymmetric encryption (RSA) to securely transmit the symmetric key. In addition, digital signatures are used to ensure data integrity and authenticate the sender.



The process works as follows:

- **File Encryption:** The sender generates a symmetric AES key and uses it to encrypt the file. AES operates in GCM mode to prevent any recognizable patterns from appearing in the encrypted data.
- **Key Encryption:** The sender encrypts the symmetric AES key using the receiver's RSA public key. The encrypted key is sent along with the encrypted file.
- **File Decryption:** Upon receiving the data, the receiver uses their RSA private key to decrypt the symmetric AES key, and then uses this key to decrypt the file.

Digital Signature and System Overview: To ensure file integrity and authenticity, the sender hashes the file and signs the hash with their private key. The receiver verifies the file by comparing the hash with the signature using the sender's public key. This system demonstrates hybrid encryption for secure and efficient file exchange, supporting multiple file types while preserving data confidentiality and integrity.

2 System Architecture

The encryption process is designed to securely encrypt and exchange files between a sender and a receiver. It involves the following steps:

1. Symmetric AES Key Generation: A 256-bit AES key is randomly generated to serve as the symmetric key for file encryption. This key enables fast and efficient encryption of large files.
2. File Encryption with AES: The file is read in binary mode and encrypted using AES in GCM mode. GCM ensures that no predictable patterns appear in the encrypted data while also providing authentication. A randomly generated initialization vector is prepended to the encrypted file.
3. Hashing and Signing the File: To ensure integrity and authenticity, the sender computes a SHA-256 hash of the file and signs it using their RSA private key. The signature is attached to the encrypted file for verification during decryption.

The decryption process allows the receiver to retrieve the original file and verify its integrity. It involves the following steps:

1. RSA encryption of the Symmetric Key: The receiver uses their RSA private key to decrypt the symmetric AES key, which was encrypted by the sender using the receiver's RSA public key.
2. File Decryption with AES: The receiver uses the decrypted AES key to decrypt the file. The decryption process employs the same GCM mode and initialization vector that were used during encryption
3. Verification: Upon decrypting the file, the receiver computes its hash and compares it with the signature decrypted using the sender's RSA public key. If the hashes match, the file is confirmed to be intact and authentic; otherwise, it is considered corrupted or tampered with.

3 Mechanisms

3.1 Key Encryption

The system uses RSA with a 2048-bit key to securely exchange the symmetric AES key between the sender and receiver. The 2048-bit RSA key provides a strong balance between security and performance. Using public-key encryption, the sender encrypts the AES key with the receiver's public key, ensuring that only the receiver, possessing the corresponding private key, can decrypt it.

3.2 File Encryption

The system uses AES with a 256-bit key (32 bytes) for file encryption. AES-256 is widely recognized for its high security and is one of the most commonly used symmetric encryption algorithms in modern cryptography.
Encryption is performed in GCM mode, which not only prevents any discernible patterns in the encrypted data but also provides built-in authentication, ensuring both confidentiality and integrity. This approach works for files of arbitrary length and type.

3.3 File Integrity and Authenticity

To ensure the file's integrity and authenticity, the system uses SHA-256 as the hash algorithm. SHA-256 is well-known for its collision resistance and security, making it suitable for generating a secure hash of the file prior to signing.
The sender signs the hash using their RSA private key, creating a digital signature. This signature allows the receiver to verify both the integrity of the file (ensuring it has not been tampered with) and its authenticity (confirming it was sent by the sender).

4 Workflow

4.1 Generate RSA Key Pairs

Generate a key pair for the sender:

```
python keyGen.py  --output_public_key_file=keys/sender_pub_key.pub
                  --output_private_key_file=keys/sender_private_key.key
                  --bits=2048
```

Generate a key pair for the receiver:

```
python keyGen.py  --output_public_key_file=keys/receiver_pub_key.pub
                  --output_private_key_file=keys/receiver_private_key.key
                  --bits=2048
```

Parameters:

`--output_public_key_file`: Path to save the public key file

`--output_private_key_file`: Path to save the private key file

`--bits`: RSA key length in bits (default: 2048, options: 1024, 2048, 4096)

4.2 Encrypt and Sign a File

Encrypt a file using the receiver's public key and sign it with the sender's private key:

```
python encryptor.py --receiver_pub_key=keys/receiver_pub_key.pub
                    --sender_private_key=keys/sender_private_key.key
                    --input_file=sample/file2.txt
                    --output_encrypted_file=payload/encrypted_file.bin
                    --output_encrypted_symmetric_key=payload/encrypted_key.key
                    --output_signature=payload/output_signature.sig
```

Parameters:

`--receiver_pub_key`: Path to receiver's public key

`--sender_private_key`: Path to sender's private key

`--input_file`: File to encrypt

`--output_encrypted_file`: Output encrypted data

`--output_encrypted_symmetric_key`: Output encrypted AES key

`--output_signature`: Output digital signature

Output Files:

`encrypted_file.bin`: Encrypted file contents

`encrypted_key.key`: Encrypted AES symmetric key

`output_signature.sig`: Digital signature for authentication

4.3 Decrypt and Verify a File

Decrypt a file using the receiver's private key and verify the sender's signature:

```
python decryptor.py --receiver_private_key=keys/receiver_private_key.key
                    --sender_pub_key=keys/sender_pub_key.pub
                    --encrypted_key=payload/encrypted_key.key
                    --input_file=payload/encrypted_file.bin
                    --output_decrypted_file=results/output_decrypted_file.txt
                    --input_signature=payload/output_signature.sig
```

Parameters:

`--receiver_private_key`: Path to receiver's private key

`--sender_pub_key`: Path to sender's public key

`--encrypted_key`: Encrypted AES key

`--input_file`: Encrypted file

`--output_decrypted_file`: Output decrypted file

`--input_signature`: Digital signature path

Output:

`output_decrypted_file.txt`: The decrypted file

Console message confirming signature verification results

5 Conclusion

In this lab, a hybrid encryption system was successfully implemented to securely exchange files between a sender and a receiver. The system combines symmetric AES-256 encryption in GCM mode for fast and secure file encryption with RSA-2048 public-key encryption to securely transmit the symmetric key. Additionally, SHA-256 hashing and RSA-based digital signatures ensure file integrity and authenticity, allowing the receiver to verify that the file has not been tampered with and confirming the sender's identity.

The workflow demonstrates the practical application of cryptographic mechanisms, supporting multiple file types while maintaining confidentiality, integrity, and authenticity. This lab highlights the effectiveness of combining symmetric and asymmetric encryption along with digital signatures for secure and reliable file exchange.

References

[1] PyCryptodome Documentation,
https://www.pycryptodome.org/src/introduction

[2] RSA Laboratories, PKCS #1 v2.2: RSA Cryptography Standard,
https://datatracker.ietf.org/doc/html/rfc8017

[3] National Institute of Standards and Technology (NIST), FIPS PUB 197: Advanced Encryption Standard (AES),
https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf

[4] National Institute of Standards and Technology (NIST), SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM),
https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf

[5] National Institute of Standards and Technology (NIST), FIPS PUB 180-4: Secure Hash Standard (SHS),
https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf